HWSEC OSS FRAMEWORK:
A Unified Flow for Open-Source Digital Design

Pablo Navarro Torrero

navarro@imse-cnm.csic.es

Instituto de Microelectrónica de Sevilla

July 4, 2025

Outline

- **PART 1: Introduction to Digital Design**
  - Hardware vs Software
  - From Code to Silicon: sim, synth, pnr

- **PART 2: The HWSEC OSS Framework**
  - Introduction & Structure
  - Live Demo: From RTL to Blinking Lights!

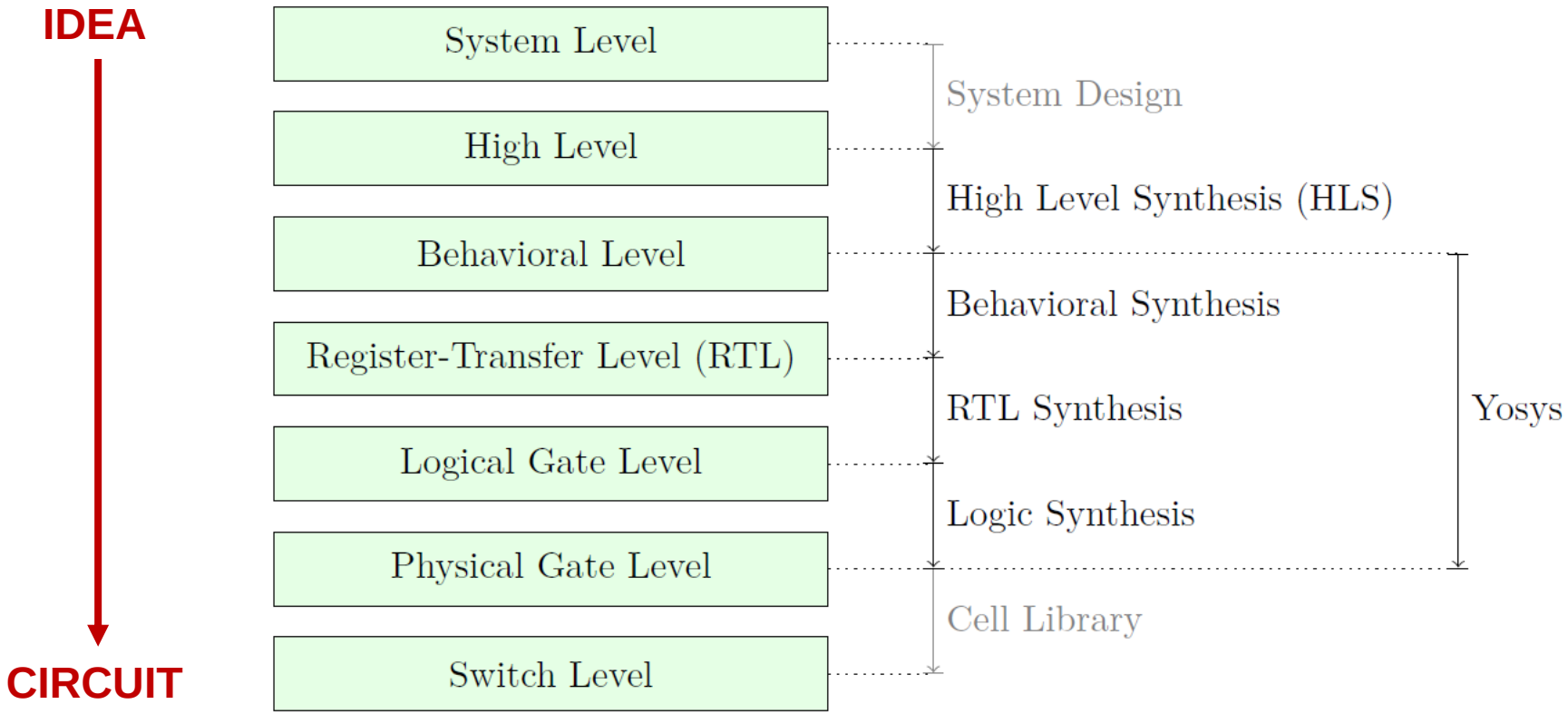
Why Digital Circuits? HW vs SW approach

	SOFTWARE	HARDWARE
Languages:	C/C++, Python, Java	Verilog, VHDL, Chisel
Paradigm:	Sequential execution	Spatial Concurrency
Advantages:	Flexibility, portability, low-cost	Parallelism, high performance, cycle accuracy, power efficiency
Disadvantages:	Limited parallelism, less performance, non cycle accuracy	Less flexible or inflexible, high cost, complexity

Algorithm: Image Sharpening Filter

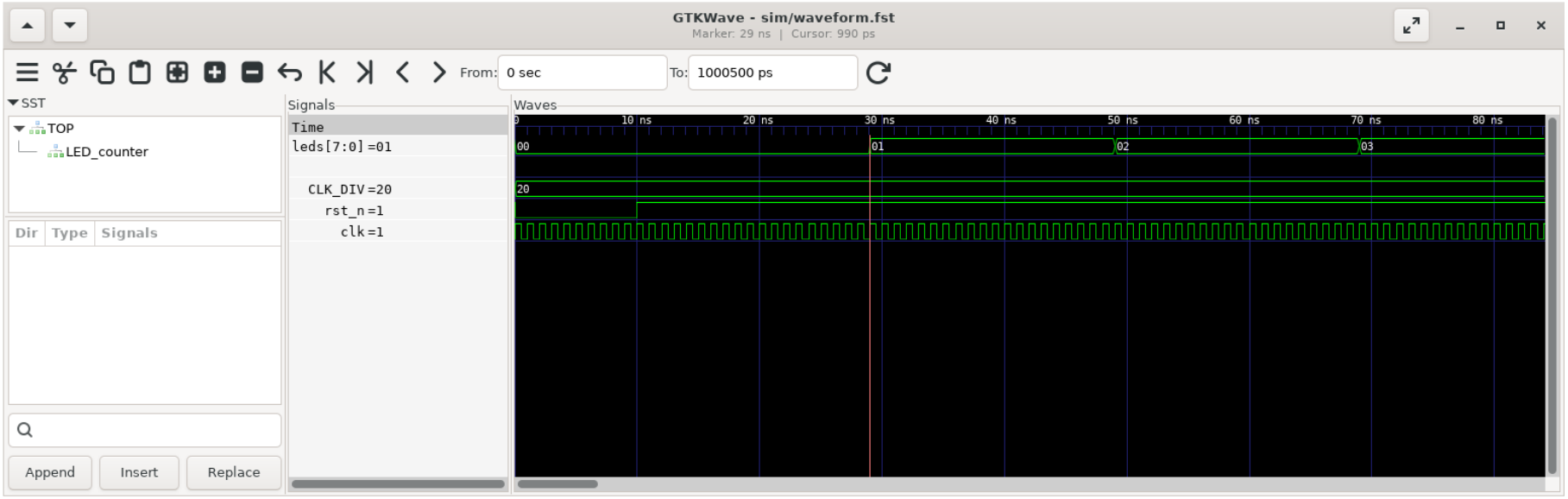
```
for (int y = 1; y < H-1; y++) {
  for (int x = 1; x < W-1; x++) {
    // For each pixel, do the math
    int sum = 0;
    for (int ky = -1; ky <= 1; ky++) {
      for (int kx = -1; kx <= 1; kx++) {
        // One operation at a time...
        sum += image[y+ky][x+kx] * kernel[ky+1][kx+1];
      }
    }
    output[y][x] = sum;
  }
}
```

The Digital Design Flow: Levels of Abstraction



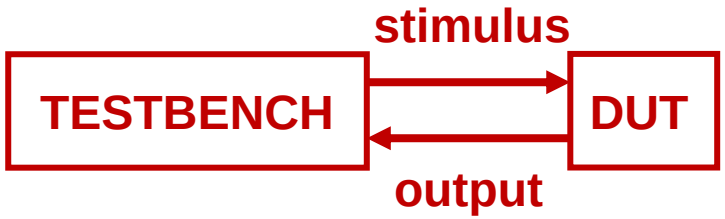
Step 1: Simulation - "Does My Code Work?"

Verify behaviour before building!



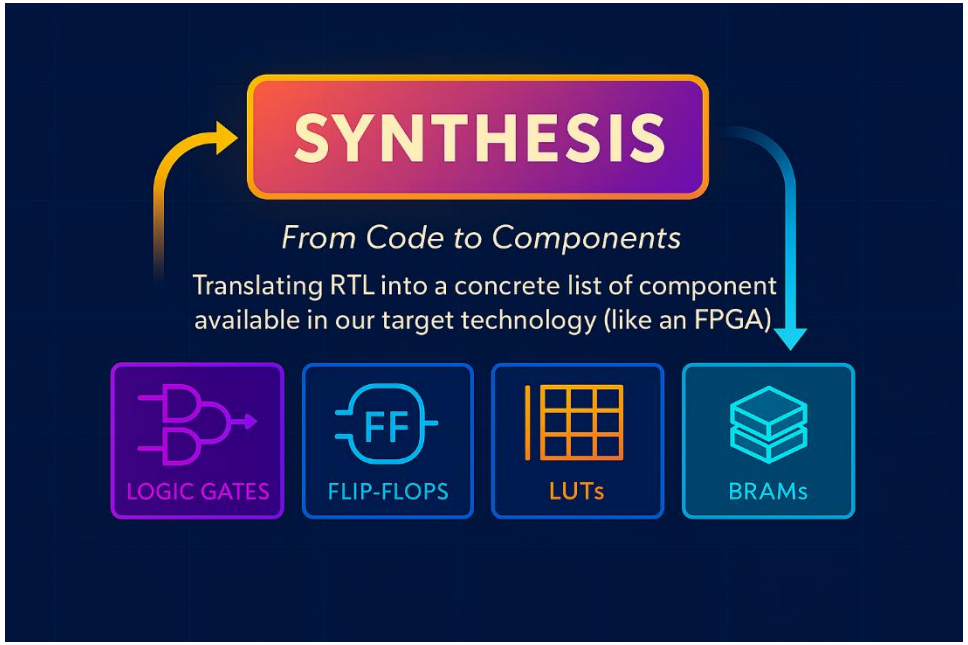
What is a Testbench?

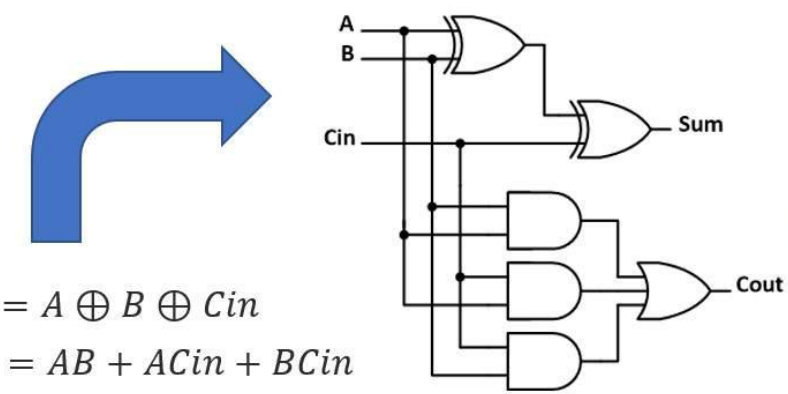
- It drives inputs into our design (DUT)
- Clocks, resets, and custom stimulus vectors
- Monitors outputs
- Reports pass/fail status



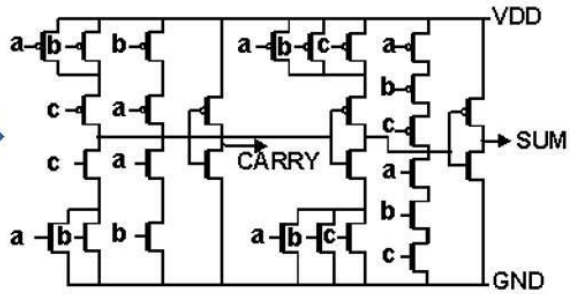
Step 2: Synthesis - "From Code to Components"

Translate RTL into logic gates and components!



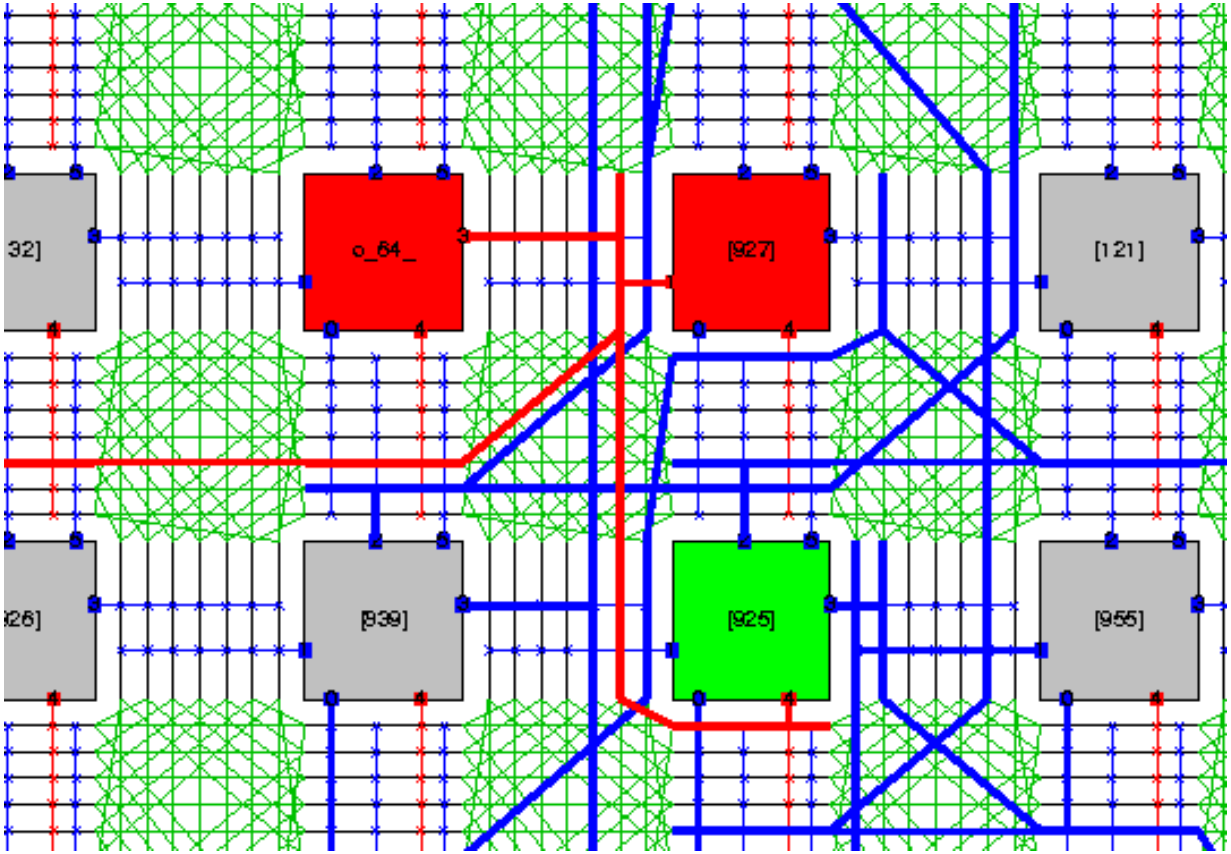


- $Sum = A \oplus B \oplus Cin$
- $Cout = AB + ACin + BCin$



Step 3: Place & Route – “Building the Circuit”

Arranging and connecting the Components!

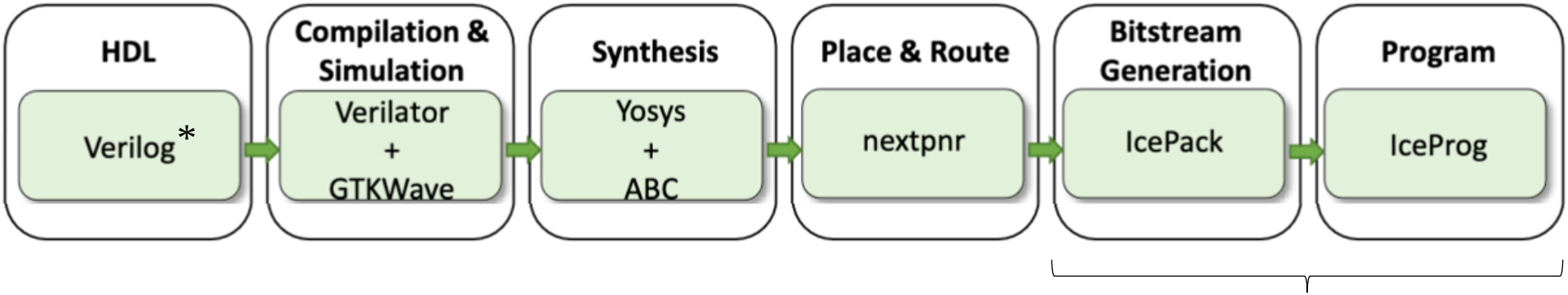


With a netlist in hand, place & route assigns each primitive to a physical location and weaves routing wires to satisfy timing and resource constraints

The Open-Source FPGA Toolchain

Verilator:
https://github.com/verilator/verilator
GTKWave:
https://github.com/gtkwave/gtkwave
Yosys:
https://github.com/YosysHQ/yosys

ABC:
https://github.com/berkeley-abc/abc
nextpnr:
https://github.com/YosysHQ/nextpnr



***Yosys + GHDL Plugin => VHDL Support**
https://github.com/ghdl/ghdl-yosys-plugin

IceStorm Project
https://clifford.at/icestorm

The HWSEC OSS Framework

H **HWSEC_OSS_FRAMEWORK**

Star 0 | Fork 0

hwsec_oss_framew… | hwsec_verilator / | + | Find file | Edit | Code

Update file settings.json
Pablo Navarro Torrero authored 2 hours ago

0f617c15 | History

Name	Last commit	Last update
fw	Minor fixes in the Makefiles and simpler...	2 hours ago
rtl	Minor fixes in the Makefiles and simpler...	2 hours ago
sim	Minor fixes in the Makefiles and simpler...	2 hours ago
Makefile	Minor fixes in the Makefiles and simpler...	2 hours ago
README.md	Minor fixes in the Makefiles and simpler...	2 hours ago
config	Minor fixes in the Makefiles and simpler...	2 hours ago

README.md

HWSEC OSS FRAMEWORK

Welcome to the HWSEC OSS Framework – a comprehensive open source framework for hardware security research. This repository integrates a complete digital design flow, from simulation to FPGA implementation, providing a unified environment to model, test, and deploy hardware designs.

Project information

- **21** Commits
- **4** Branches
- **0** Tags
- **452 MiB** Project Storage

- README
- + Add LICENSE
- + Add CHANGELOG
- + Add CONTRIBUTING
- + Enable Auto DevOps
- + Add Kubernetes cluster
- + Set up CI/CD
- + Add Wiki
- + Configure Integrations

Created on
April 15, 2025

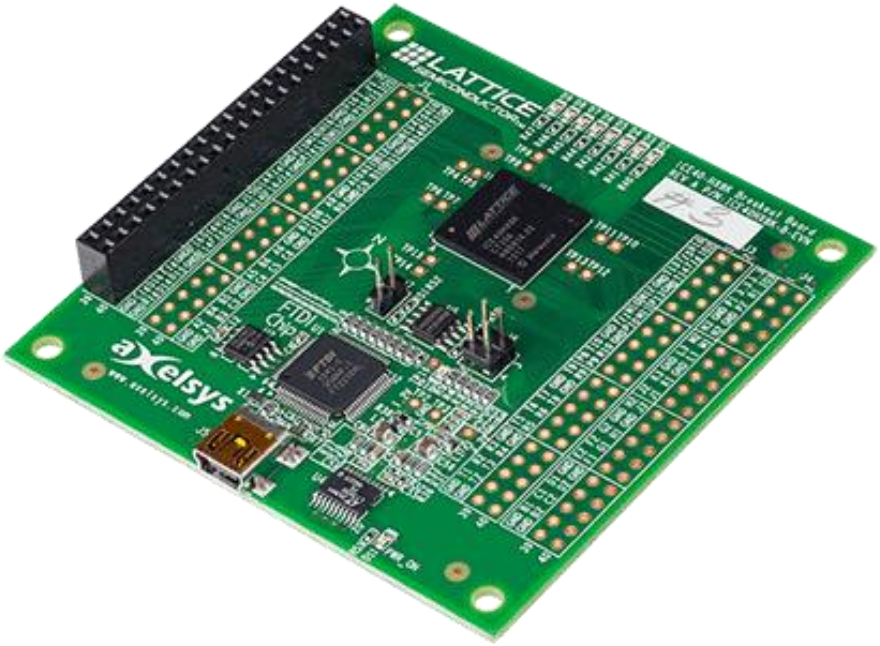
https://gitlab.com/hwsec/hwsec_verilator

HWSEC OSS Framework: Repository Structure

- fw
  - bin
  - ld
    - riscv.ld
  - src
    - firmware.c
    - start.s
  - Makefile
- pnr
- prog
- rtl
  - LED_counter.pcf
  - LED_counter.v
- sim
  - src
    - config
    - obj_dir
    - LED_counter_tb.cpp
    - sim_utils.cpp
    - sim_utils.h
  - waveform.gtkw
- synth
- Makefile
- README.md

```
.
├── fw                          # Firmware code for RISC-V bare-metal targets
│   ├── Makefile                # Firmware build rules
│   ├── bin                     # Compiled firmware binaries
│   ├── ld
│   │   └── riscv.ld            # Linker script
│   └── src                     # Firmware source code
│       ├── firmware.c
│       └── start.s
├── pnr                         # Place-and-Route outputs
├── prog                        # FPGA programming files
├── rtl                         # RTL (Verilog/SystemVerilog) sources and const
│   ├── LED_counter.pcf         # FPGA pin constraints
│   └── LED_counter.v           # Top module (example)
├── sim                         # Simulation setup
│   ├── src
│   │   ├── config              # Configuration program to update design parame
│   │   ├── LED_counter_tb.cpp  # Demo C++ testbench (to be updated/modified fo
│   │   ├── sim_utils.cpp       # Auxiliary simulation functions.
│   │   └── sim_utils.h         # Declarations for simulation utilities.
│   └── waveform.gtkw           # Optional GTKWave session file.
├── synth                       # Synthesis outputs: JSON files, synthesis logs
├── Makefile                    # Main Makefile for all workflows
└── README.md                   # This documentation
```

Live Demo:
From RTL to Blinking Lights



Live Demo: Prerequisites

Prerequisites

Install these tools to use the framework:

1. **Verilator (v5.035):** Verilog/SystemVerilog simulator.
2. **GTKWave (v3.3.116):** Waveform visualization tool.
3. **Yosys (v0.51):** RTL synthesis framework.
4. **Nextpnr (v0.6):** Vendor-neutral FPGA place-and-route tool.
5. **Project Icestorm:** Tools for Lattice iCE40 FPGA bitstream generation.
6. **RISC-V GNU toolchain (v14.2.0):** RISC-V C/C++ cross-compiler for firmware.

Live Demo: Download the Repository

Name	Date modified	Type	Size
.git	03/07/2025 20:21	File folder	
fw	03/07/2025 20:21	File folder	
pnr	03/07/2025 20:21	File folder	
prog	03/07/2025 20:21	File folder	
rtl	03/07/2025 20:21	File folder	
sim	03/07/2025 20:21	File folder	
synth	03/07/2025 20:21	File folder	
Makefile	03/07/2025 20:21	File	6 KB
README.md	03/07/2025 20:21	Archivo de origen ...	10 KB

```
navarro@pcnavarro:~/Desktop/tutorial$ git clone --single-branch https://gitlab.com/hwsec/hwsec_verilator
```

Live Demo: Configure the Repository

1. **Replace RTL Files With Your Own:**
   Place your Verilog/SystemVerilog source files in the **rtl** folder. The provided demo files (e.g., `LED_counter.v`) are provided only as examples and can be overwritten.

   > *Note:* The top module file must be named according to the top module. For example, if the top module name is `LED_counter`, the Verilog/SystemVerilog file must be named `LED_counter.v`.

2. **Configure the Environment:**
   Type `make config` to compile the `config` program and execute it to update the source files and the Makefile with your design parameters (top module name and clock frequency).

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make config
g++ -Wall -O2 sim/src/config/config.cpp -o config -lstdc++fs
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ ./config
Error: Missing required flags.
Usage: ./config -TOP_MODULE=<module_name> -CLOCK_FREQUENCY=<number> [-v]

Mandatory flags:
  -TOP_MODULE=<module_name>    Specifies the design top module.
  -CLOCK_FREQUENCY=<number>    Specifies the clock frequency in MHz as an integer (> 0).

Optional flags:
  -v                           Verbose output (print file update messages).
  -h, --help                   Show this help message.
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ ./config -TOP_MODULE=LED_counter -CLOCK_FREQUENCY=1000
Configuring design with:
  Top Module: LED_counter
  Clock Frequency: 1000
```

Live Demo: LED Counter Module

- rtl
  - LED_counter.pcf
  - LED_counter.v

```
module LED_counter #(
                    parameter CLK_DIV = 20          //-- Clock Divisor
                    )
                    (
                     input  wire        clk,        //-- Clock Signal
                     input  wire        rst_n,      //-- Active Low Reset
                     output reg [7:0]   leds        //-- LEDs
                     );

    reg [$clog2(CLK_DIV)-1:0] div_counter /*verilator public*/;     // Makes visible this signal during simulation

    always @(posedge clk) begin
        if (!rst_n) begin
            div_counter <= 0;
            leds        <= 0;
        end
        else if (div_counter == CLK_DIV - 1) begin
            div_counter <= 0;
            leds        <= leds + 1;
        end
        else begin
            div_counter <= div_counter + 1;
            leds        <= leds;
        end
    end

endmodule
```

Live Demo: Lint your Code!

NO ERROR NO WARNINGS COMPILATION

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make lint
verilator --lint-only -Wall --top-module LED_counter   rtl/LED_counter.v   -I/home/navarro/Desktop/FPGA/yosys/techlibs/ice40
- V e r i l a t i o n   R e p o r t: Verilator 5.035 devel rev v5.034-154-g6bb57e463
- Verilator: Built from 0.023 MB sources in 2 modules, into 0.014 MB in 7 C++ files needing 0.000 MB
- Verilator: Walltime 0.003 s (elab=0.000, cvt=0.001, bld=0.000); cpu 0.003 s on 1 threads; alloced 20.129 MB
```

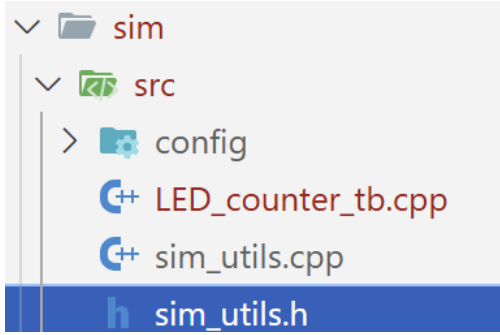
WARNING COMPILATION

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make lint
verilator --lint-only -Wall --top-module LED_counter   rtl/LED_counter.v   -I/home/navarro/Desktop/FPGA/yosys/techlibs/ice40
%Warning-UNUSEDSIGNAL: rtl/LED_counter.v:34:9: Signal is not driven, nor used: 'en_div'
                                             : ... note: In instance 'LED_counter'
   34 |     reg en_div;
      |         ^~~~~~
                       ... For warning description see https://verilator.org/warn/UNUSEDSIGNAL?v=5.035
                       ... Use "/* verilator lint_off UNUSEDSIGNAL */" and lint_on around source to disable this message.
%Error: Exiting due to 1 warning(s)
make: *** [Makefile:91: lint] Error 1
```

ERROR COMPILATION

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make lint
verilator --lint-only -Wall --top-module LED_counter   rtl/LED_counter.v   -I/home/navarro/Desktop/FPGA/yosys/techlibs/ice40
%Error-PROCASSWIRE: rtl/LED_counter.v:39:13: Procedural assignment to wire, perhaps intended var (IEEE 1800-2023 6.5): 'leds'
                                           : ... note: In instance 'LED_counter'
   39 |             leds        <= 0;
      |             ^~~~
                     ... For error description see https://verilator.org/warn/PROCASSWIRE?v=5.035
%Error-PROCASSWIRE: rtl/LED_counter.v:43:13: Procedural assignment to wire, perhaps intended var (IEEE 1800-2023 6.5): 'leds'
                                           : ... note: In instance 'LED_counter'
   43 |             leds        <= leds + 1;
      |             ^~~~
%Error-PROCASSWIRE: rtl/LED_counter.v:47:13: Procedural assignment to wire, perhaps intended var (IEEE 1800-2023 6.5): 'leds'
                                           : ... note: In instance 'LED_counter'
   47 |             leds        <= leds;
      |             ^~~~
%Error: Exiting due to 3 error(s)
make: *** [Makefile:91: lint] Error 1
```

Live Demo: Configure your Testbench!

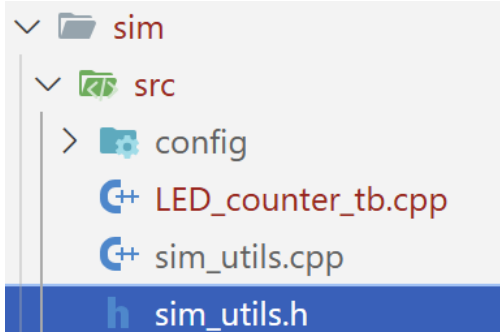



```
// Design Top Module
#include "VLED_counter.h"
// Internal Signals (/*verilator public*/)
#include "VLED_counter_LED_counter.h"

//-----------------------------------------------------------------------------
// Waveform Configuration
//-----------------------------------------------------------------------------
#define TRACE_SIGNALS   true
#define DEPTH_LEVELS    5

//-----------------------------------------------------------------------------
// Clock Timing Parameters
//-----------------------------------------------------------------------------
// Clock Frequency (in MHz)
#define CLOCK_FREQ      1000
// Clock Period (in ns)
#define CLOCK_PERIOD    (1000 / CLOCK_FREQ)
// Time precision is 100ps --> 10 time units is one clock cycle
#define HALF_CYCLE      5 * CLOCK_PERIOD
// Total simulation duration (in Clock Cycles)
#define MAX_SIM_TIME    1000
```

Live Demo: Verilog Functions!



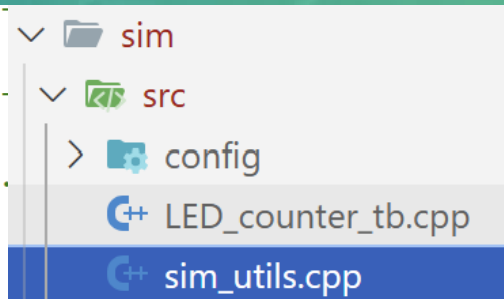

```
//-----------------------------------------------------------------------------
// Verilog-like $display and $monitor Prototypes
//-----------------------------------------------------------------------------
void verilog_display(bool prepend_time, const char* format, ...);
void verilog_monitor(const char* format, int current_value, int* last_value);

//-----------------------------------------------------------------------------
// Simulation Control Prototypes
//-----------------------------------------------------------------------------
void sim_step(VLED_counter* dut, VerilatedFstC* m_trace);
void verilog_delay(vluint64_t delay, VLED_counter* dut, VerilatedFstC* m_trace);
```

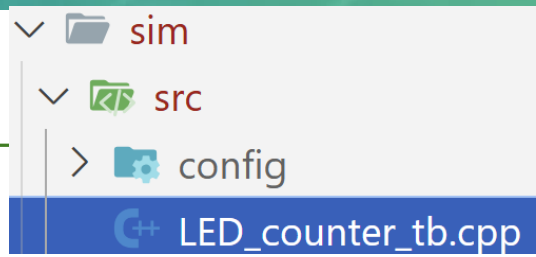
Live Demo: Monitor a Signal!

```
//-----------------------------------------------------------------------------
// Verilog $monitor equivalent function
//-----------------------------------------------------------------------------
// A simple monitor function that checks if a signal's current value differs
// from the stored value. If so, it prints the change and updates the stored value.
// The 'format' string is used to print the value—for example, you can pass "out =
void verilog_monitor(const char* format, int current_value, int* last_value) {
    if (current_value != *last_value) {
        verilog_display(true, format, current_value);
        *last_value = current_value;
    }
}
// Define the variables to monitorize
int monitor_leds = 0;

//-----------------------------------------------------------------------------------------
// Single simulation step: toggle clk, evaluate DUT, dump FST trace,
// and advance simulation time by one half cycle
//-----------------------------------------------------------------------------------------
void sim_step(VLED_counter* dut, VerilatedFstC* m_trace) {
    // Toggle clock.
    dut->clk = !dut->clk;
    // Evaluate the design model
    dut->eval();
    // Dump simulation data for FST trace (if needed)
    if (TRACE_SIGNALS) m_trace->dump(sim_time);
    // Check monitored signals
    verilog_monitor("leds = %d", dut->leds, &monitor_leds);
    // Increase simulation time by the half cycle.
    sim_time += HALF_CYCLE;
```



Live Demo: LED Counter Testbench




```
//-----------------------------------------------------------
// Test Values
//-----------------------------------------------------------

dut->rst_n = 0;
verilog_delay(10, dut, m_trace);
dut->rst_n = 1;

// Simulate until max simulation time is reached
while (sim_time <= 2*HALF_CYCLE*MAX_SIM_TIME)
{
    verilog_display(false, "div_counter = %d", dut->LED_counter->div_counter);
    if ((dut->LED_counter->div_counter) == 0) verilog_display(false, "\n");

    sim_step(dut, m_trace);
    sim_step(dut, m_trace);
}
```

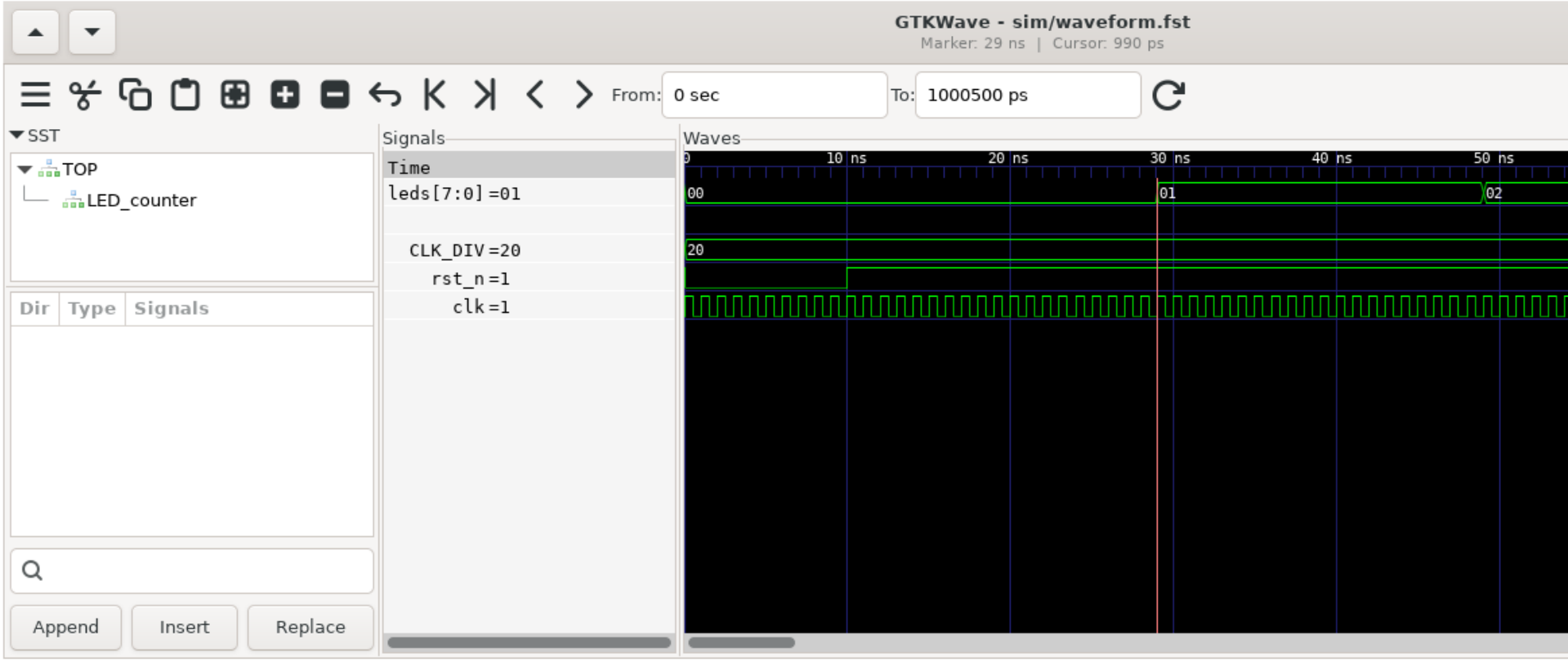
Live Demo: Simulation! (Console Output)

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make sim
div_counter = 19
[t = 989 ns] leds = 49
div_counter = 0


div_counter = 1
div_counter = 2
div_counter = 3
div_counter = 4
div_counter = 5
div_counter = 6
div_counter = 7
div_counter = 8
div_counter = 9
div_counter = 10
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$
```

Live Demo: Simulation! (Waveform)

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make waves
```



Live Demo: Synthesis!

- synth
  - LED_counter.json
  - LED_counter.log
  - stat.txt

```
=== LED_counter ===

   Number of wires:                 29
   Number of wire bits:             43
   Number of public wires:          29
   Number of public wire bits:      43
   Number of ports:                  3
   Number of port bits:             10
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                 38
     SB_CARRY                        9
     SB_DFFESR                       8
     SB_DFFSR                        5
     SB_LUT4                        16
```

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make synth-ice40
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make synth-xilinx
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make synth-generic
```

Live Demo: Place & Route! (Constraints)

rtl

LED_counter.pcf

```
# Pinout for the iCE40-HX8K Breakout Board

set_io clk      J3
set_io rst_n    C16 -pullup yes

#set_io ser_tx   B12
#set_io ser_rx   B10

set_io leds[7]  B5  # D9
set_io leds[6]  B4  # D8
set_io leds[5]  A2  # D7
set_io leds[4]  A1  # D6
set_io leds[3]  C5  # D5
set_io leds[2]  C4  # D4
set_io leds[1]  B3  # D3
set_io leds[0]  C3  # D2
```

Live Demo: Place & Route!

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make nextpnr-ice40
```

```
Info: Program finished normally.
icetime -p rtl/LED_counter.pcf -P ct256 -r pnr/LED_count
// Reading input .pcf file..
// Reading input .asc file..
// Reading 8k chipdb file..
// Creating timing netlist..
// Timing estimate: 5.43 ns (184.15 MHz)
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$
```

Live Demo: Program the Board! (Config.)

1. Configure the Makefile

```
# FPGA Target
BOARD          = ice40hx8k
FPGA_VARIANT   = hx8k
FPGA_PACKAGE   = ct256
CLOCK_FREQUENCY = 12
```

2. If using WSL, attach the device using usbipd*

```
PS C:\Users\navarro> usbipd list

PS C:\Users\navarro> usbipd bind --busid X-Y

PS C:\Users\navarro> usbipd attach --wsl --busid X-Y
```

*https://github.com/dorssel/usbipd-win

Live Demo: Program the Board!

```
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$ make prog_ice40

icepack pnr/LED_counter.asc prog/LED_counter.bin
iceprog prog/LED_counter.bin
init..
cdone: low
reset..
cdone: low
flash ID: 0x20 0xBA 0x16 0x10 0x00 0x00 0x23 0x80 0x80 0x52 0x10 0x00 0x49
file size: 135100
erase 64kB sector at 0x000000..
erase 64kB sector at 0x010000..
erase 64kB sector at 0x020000..
programming..
done.
reading..
VERIFY OK
cdone: high
Bye.
navarro@pcnavarro:~/Desktop/tutorial/hwsec_verilator$
```

HWSEC OSS FRAMEWORK

Pablo Navarro Torrero

navarro@imse-cnm.csic.es

Instituto de Microelectrónica de Sevilla

July 4, 2025